

# Secure Admin Panel (Node.js + Express + JWT)

## Folder Structure

```
admin-panel/  
|  
├─ server.js  
├─ package.json  
├─ .env  
├─ middleware/  
|   └─ auth.js  
├─ routes/  
|   └─ admin.js  
└─ public/  
    ├─ login.html  
    ├─ dashboard.html  
    ├─ style.css  
    └─ app.js
```

## package.json

```
{  
  "name": "secure-admin-panel",  
  "version": "1.0.0",  
  "main": "server.js",  
  "scripts": {  
    "start": "node server.js",  
    "dev": "nodemon server.js"  
  },  
  "dependencies": {  
    "bcryptjs": "^2.4.3",  
    "cookie-parser": "^1.4.6",  
    "cors": "^2.8.5",  
    "dotenv": "^16.4.5",  
    "express": "^4.19.2",  
    "helmet": "^7.1.0",  
    "jsonwebtoken": "^9.0.2"  
  },  
  "devDependencies": {  
    "nodemon": "^3.1.0"  
  }  
}
```

```
}  
}
```

## 2 .env

```
PORT=5000  
JWT_SECRET=super_secure_secret_change_this  
ADMIN_USERNAME=ameerhamzamayo  
ADMIN_PASSWORD=ameerhamzamayo
```

⚠ Real production project me `.env` ko GitHub par upload mat karna.

## 3 server.js

```
require("dotenv").config();  
  
const express = require("express");  
const path = require("path");  
const bcrypt = require("bcryptjs");  
const jwt = require("jsonwebtoken");  
const helmet = require("helmet");  
const cors = require("cors");  
const cookieParser = require("cookie-parser");  
  
const app = express();  
  
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));  
app.use(cookieParser());  
app.use(cors());  
app.use(helmet());  
app.use(express.static("public"));  
  
const PORT = process.env.PORT || 5000;  
  
const ADMIN_USERNAME = process.env.ADMIN_USERNAME;  
const ADMIN_PASSWORD = process.env.ADMIN_PASSWORD;  
  
// Login API  
app.post("/api/login", async (req, res) => {  
  try {
```

```

const { username, password } = req.body;

if (!username || !password) {
  return res.status(400).json({
    success: false,
    message: "Username and password required"
  });
}

if (username !== ADMIN_USERNAME || password !== ADMIN_PASSWORD) {
  return res.status(401).json({
    success: false,
    message: "Invalid credentials"
  });
}

const token = jwt.sign(
  { username },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);

res.cookie("token", token, {
  httpOnly: true,
  secure: false,
  sameSite: "strict",
  maxAge: 24 * 60 * 60 * 1000
});

return res.json({
  success: true,
  message: "Login successful"
});
} catch (error) {
  console.error(error);

  res.status(500).json({
    success: false,
    message: "Server error"
  });
}
});

// Middleware
function auth(req, res, next) {
  const token = req.cookies.token;

  if (!token) {

```

```

    return res.redirect("/login.html");
  }

  try {
    jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.redirect("/login.html");
  }
}

// Protected Route
app.get("/dashboard", auth, (req, res) => {
  res.sendFile(path.join(__dirname, "public", "dashboard.html"));
});

// Logout
app.get("/logout", (req, res) => {
  res.clearCookie("token");
  res.redirect("/login.html");
});

// Default Route
app.get("/", (req, res) => {
  res.redirect("/login.html");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```

## 4 public/login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Login</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

```

```

<div class="login-box">
  <h2>Admin Panel</h2>

  <form id="loginForm">
    <input type="text" id="username" placeholder="Username" required>
    <input type="password" id="password" placeholder="Password" required>
    <button type="submit">Login</button>
  </form>

  <p id="message"></p>
</div>
</div>

<script src="app.js"></script>
</body>
</html>

```

## 5 public/dashboard.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="dashboard">
  <h1>Welcome Ameer Hamza 🙌</h1>

  <div class="card-grid">
    <div class="card">
      <h2>Total Users</h2>
      <p>1250</p>
    </div>

    <div class="card">
      <h2>Total Posts</h2>
      <p>874</p>
    </div>

    <div class="card">

```

```

        <h2>Messages</h2>
        <p>532</p>
    </div>
</div>

    <a class="logout-btn" href="/logout">Logout</a>
</div>

</body>
</html>

```

## 6 public/style.css

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
}

body {
    background: #1877f2;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}

.container {
    width: 100%;
    display: flex;
    justify-content: center;
    align-items: center;
}

.login-box {
    width: 350px;
    background: white;
    padding: 40px;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}

.login-box h2 {

```

```

    text-align: center;
    margin-bottom: 25px;
    color: #1877f2;
}

.login-box input {
    width: 100%;
    padding: 12px;
    margin-bottom: 15px;
    border: 1px solid #ddd;
    border-radius: 8px;
}

.login-box button {
    width: 100%;
    padding: 12px;
    background: #1877f2;
    border: none;
    color: white;
    border-radius: 8px;
    cursor: pointer;
    font-size: 16px;
}

.dashboard {
    width: 90%;
    max-width: 1000px;
    background: white;
    padding: 30px;
    border-radius: 20px;
}

.card-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
    margin-top: 30px;
}

.card {
    background: #f5f7fb;
    padding: 25px;
    border-radius: 15px;
    text-align: center;
}

.logout-btn {
    display: inline-block;

```

```
margin-top: 30px;
background: red;
color: white;
padding: 12px 20px;
text-decoration: none;
border-radius: 10px;
}
```

## 7 public/app.js

```
const form = document.getElementById("loginForm");
const message = document.getElementById("message");

form.addEventListener("submit", async (e) => {
  e.preventDefault();

  const username = document.getElementById("username").value;
  const password = document.getElementById("password").value;

  const response = await fetch("/api/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ username, password })
  });

  const data = await response.json();

  if (data.success) {
    window.location.href = "/dashboard";
  } else {
    message.innerText = data.message;
    message.style.color = "red";
  }
});
```





## Install & Run

### Install Packages

```
npm install
```

### Start Server

```
npm run dev
```



## Login Credentials

Username: ameerhamzamayo

Password: ameerhamzamayo



## Included Security Improvements

- Helmet Security
- JWT Authentication
- Secure Cookies
- Protected Dashboard Route
- Environment Variables
- Modular Structure
- Responsive Design
- Production Ready Base



## Future Upgrades

- MongoDB Database
- Real User Management
- Analytics Charts
- AI Moderation
- Notifications System
- Live Chat
- Admin Settings

- Dark Mode
- Multi Admin Support